"Evaluación integradora - Programación para Inteligencia Artificial - Segunda parte".

Estudiante: Gentile, Agutín Raúl
DNI: 32,7097,735
Fecha entrega: 4/09/2026
Legajo: 199433

Gentile, Agustín Raúl (Dni: 32709735) (Legajo: 199433)

1. Programación orientada al objeto:

a) Explique qué es una clase y qué es un objeto. ¿Cuál es la relación entre ambos? De un ejemplo concreto de una clase y cómo crear objetos a partir de ella.

Una “clase” es un molde/una plantilla, la cual define la estructura (atributos) y el comportamiento (métodos) de un tipo de objeto. El “objeto” es una instancia concreta de una determinada clase, que define valores específicos en sus atributos. La relación entre ambos conceptos es que la clase nos sirve como base para crear objetos, por si sola la clase no se puede utilizar, debemos instanciarla y crear un objeto del tipo de objeto que define la clase.

```
 class Libro:
   def __init__(self, titulo, autor, anio):
      self.titulo = titulo
      self.autor = autor
      self.anio = anio
      self.disponible = True


   def prestar(self):
      if(self.disponible):
         self.disponible = False
      else:
         print("Eror no disponible")
         return None

   def regresar(self):
      self.disponible = True

   def esta_disponible(self):
      return self.disponible

   def mostrar_info(self):
      print(f"Datode del libro son: Nombre: {self.titulo}, autor: {self.autor} año: {self.anio}")

libro1 = Libro("Orgullo y prejuicio", "Jane Austen", 1998)
libro2 = Libro("Crimen y castigo", "Fiódor Dostoyevski", 2001)
libro3 = Libro("La Odisea", "Homero", 1954)
libro4 = Libro("El Señor de los Anillos", "J. R. R. Tolkien",1975)
libro1.mostrar_info();
libro2.mostrar_info();
libro3.mostrar_info();
libro4.mostrar_info();
libro1.prestar()
if(libro1.esta_disponible() == False):
   print("libro NO disponible")
libro1.regresar()
if(libro1.esta_disponible()):
```

```
    print("libro disponible")
-------------------------------------------------------------------------------
```

b) ¿Qué es el encapsulamiento y por qué es importante en programación orientada a objetos? Mencione cómo se implementa en Python usando convenciones de nombres.

El encapsulamiento es uno de los principios de la POO que consiste en ocultar detalles internos de implementación de la clase, protegiendo datos sensibles, exponiendo una interfaz publica al exterior, para que puedan acceder o manipular esta información protegida. Es importante por nos permite ocultar y proteger datos sensibles. Nos permite tener más seguridad; ya que el usuario (no puede o no debería), manipular los datos directamente, solo hacerlo mediante los métodos públicos (interfaz) , abstraernos ya que debemos saber que hace cada método y no como lo hace.

Se implementa en python mediante la convención de nombres: podemos distinguir tres tipos de atributos:

Públicos: comienzan con letra minúscula, accesibles desde el exterior.

Protegido: comienzan con 1 solo guion bajo "_". Internos de la clase, no deberían accederse desde afuera de la clase

Privado: comienzan con 2 guiones bajo "__".

c) Explique qué es la herencia y cuál es su principal ventaja. De un ejemplo de una jerarquía de clases donde la herencia sea útil (por ejemplo, vehículos, animales, formas geométricas, etc.).

La herencia es una propiedad del paradigma de la POO, que nos permite crear una clase a partir de otra., heredando atributos y comportamiento. Es esencialmente útil, ya que nos permite extender una clase base, agregando mas funcionalidad o atributos específicos en la clase hija. Con la herencia nos permite mantener el software mas limpio y mantenible; si se cambia algo en la clase padre, todas las clases hijas notan este cambio.

```
class Figura:
    def __init__(self, nombre):
        self.nombre = nombre
        self.color = "Rojo"

    def info(self):
        print(f"Figura geometrica: {self.nombre} .")

class Cuadrado(Figura):
    def __init__(self, lado):
        super().__init__("Cuadrado")
        self.lado = lado

    def calcular_area(self):
        return self.lado * self.lado
```

Gentile, Agustín Raúl (Dni: 32709735) (Legajo: 199433)

2. Grafos

a). Explique qué es un grafo y cuáles son sus componentes principales (vértices y aristas). ¿Qué diferencia hay entre un grafo dirigido y uno no dirigido?

Un grafo es una estructura de datos que nos permite definir relación entre elementos: se compone de Nodos/Vértices representa a los elementos del grafo y de aristas definen las conexiones/relaciones entre los Vertices. Además podemos distinguir entre grafos ponderados; aquellos en los cuales las aristas tienen peso y los no ponderados.

La principal diferencia entre un grafo dirigido y uno no dirigido, es que los grafos dirigidos las aristas definen relación en un solo sentido(unidireccional), caso contrario al "no dirigido" (relación bidireccional).

b).Imagine que están diseñando un sistema de recomendación de amistades para una red social. Los usuarios pueden seguirse entre sí. Dibuje un grafo que represente las siguientes relaciones: Ana sigue a Carlos y Beatriz, Carlos sigue a Diego, Beatriz sigue a Diego y Elena, Diego sigue a Ana. Indique si este grafo es dirigido o no dirigido y justifique su respuesta.

Si traficamos las relaciones entre estas personas / usuarios en un grafo, los usuarios serían representados con Nodos y las aristas definen la ración entre ellos.

Ana → Carlos, Beatriz

Carlos → Diego

Beatriz → Diego, Elena

Diego → Ana

Elena → (no sigue a nadie)

Analizando el grafo, se observa que existe relaciones unidireccionales entre estos usuarios, estamos en presencia de un grafo dirigido; ya que hay relación en una sola dirección. Este grafo se representa mediante un grafo dirigido.
Para recalcar, si tomamos la parte del enunciado que dice **"Los usuarios pueden seguirse entre sí",** y esto hace mención a que un usuario puede seguir a otro y viceversa (ejem: Ana sigue a Carlos, y Carlos sigue a Ana); como existe una relación bidireccional el grafo sería no dirigido.

c).Si implementáramos este grafo en Python usando programación orientada a objetos, ¿qué clases necesitaríamos crear? Mencione qué atributos y métodos tendrían estas clases.

class Usuario:

```
# representa los nodos, guarda relacion entre el usuario y a quien sigue en un diccionario de datos
    def __init__(self, nombre):
        self.nombre = nombre
```

```
        self.seguidos = {}

    def seguir(self, nuevo_amigo):
      # Si el amigo (relación) no existe , la agrega al diccionario de seguidos
        if nuevo_amigo.nombre not in self.seguidos:
            self.seguidos[ nuevo_amigo .nombre] =   nuevo_amigo

    def dejar_de_seguir(self, amigo):
            # borra la relación con el amigo parametrizado; lo quita del diccionario de seguidos
        if amigo.nombre in self.seguidos:
            del self.seguidos[amigo.nombre]

    def obtener_seguidos(self):
            # retorna una lista de los amigos (seguidos)
        return list(self.seguidos.keys())
```

class RedSocial:

```
    def __init__(self, dirigido=True):
        self.usuarios = {}   # diccionario de datos clave: nombre, valor: objeto Usuario
        self.dirigido = dirigido

    def agregar_usuario(self, nombre):
            #agrega un determinado usuario a la red social; inicializandolo con sus relaciones a cero
        if nombre not in self.usuarios:
            self.usuarios[nombre] = Usuario(nombre)

    def agregar_relacion(self, user_origen, user_destino):
```

#agrega una relación (un nuevo amigo) al usuario parametrizado, verifica si lo usuarios están presente en la red (grafo) y si no, primero los inserta como nuevos usuarios a la red y despues establece su relacion

```
        if user_origen in self.usuarios and user_destino in self.usuarios:
            self.usuarios[user_origen].seguir(self.usuarios[user_destino])
            if not self.dirigido:
                self.usuarios[user_destino].seguir(self.usuarios[user_origen])

    def eliminar_relacion(self, user_origen, user_destino):
            # elimina relacion entre un user_origen y user_destino
        if user_origen in self.usuarios and user_destino in self.usuarios:
            self.usuarios[user_origen].dejar_de_seguir(self.usuarios[user_destino])
            if not self.dirigido:
                self.usuarios[user_destino].dejar_de_seguir(self.usuarios[user_origen])
```

3. *Regresión y machine learning*

a).Explique qué es la regresión en el contexto de machine learning. ¿En qué se diferencia de la clasificación? De un ejemplo concreto de un problema que se resolvería con regresión.

Tanto la regresión como la clasificación son dos técnicas del machine learning pertenecientes al aprendizaje supervisado. El objetivo de la regresión es predecir un valor numérico continuo a partir de un conjunto de valores de entrada, en cambio la clasificación su objetivo es predecir una categoría a partir de un conjunto de variables de entrada . Con la regresión predice un valor numérico, en cambio, en la clasificación se predice una etiqueta, a qué categoría pertenece la entrada.

Un ejemplo de clasificación sería clasificar correros electrónicos entre spam o no span, o clasificar si una imagen pertenece a una determinada categoria.

Un ejemplo de predicción sería predecir el precio de una casa a partir de un conjunto de características.

b).¿Qué es la normalización de datos y por qué es importante antes de entrenar modelos de machine learning? Mencione qué herramienta de Scikit-learn usaría para normalizar datos.

La normalización es una técnica que se utiliza cuando se trabaja con datos para que todos estén y trabajen en una misma escala; con esto conseguimos que a todos los datos se les de la misma importancia, a veces se trabaja con datos en distintas medidas y escalas muy variadas, si un determinado dato tiene una escala muy superior al resto, en determinados algoritmos, este dato dominaria sobre el resto; para esto se usa la normalización; busca que los datos tengan media cero

y desvío estándar uno. La herramienta que nos sirve para esto es “StandardScaler”, la cual busca media cero, y desvío estándar uno.

c).Explique qué significa dividir los datos en conjunto de entrenamiento y conjunto de prueba. ¿Por qué es importante hacer esta división? ¿Qué función de Scikit-learn usarían para hacerlo?

Dividir a los datos en dos entre entrenamiento y prueba, significa que se parte una dataset de datos en dos; se deja una parte de el mismo (el más grande) , para entrenar a los modelos, y el resto (una parte más pequeña del dataset), para las pruebas, evalua el comportamiento y desempeño del modelo.

La función que nos permite realizar está división es:

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

Explicación del análisis y desarrollo del sistema.

Para organizar el código separé la lógica en varias clases. Producto y Venta representan a las entidades del negocio y se encargan de validar sus propios datos. Inventario maneja los CSV y los DataFrames, y es la que da de alta productos, registra ventas y descuenta stock. Predicciones guarda los modelos entrenados y hace las predicciones.

El sistema muestra un menú iterativo al usuario con las distintas funcionalidades.

Me apoyo de un modelo de IA (claude) para generar los dataset iniciales de ventas.cvs y productos.csv, con el objetivo de testear el entrenamiento y predicciones. Se parte de un dataset con datos ya cargados, a los cuales se puede agregar información como nuevos productos o ventas.

En cuanto a las librerías: usé **Pandas** para trabajar con los datos como DataFrames y leer/escribir los CSV, **NumPy** para algunos cálculos estadísticos, y **scikit-learn** para entrenar un modelo de regresión lineal por producto